



CHALMERS
UNIVERSITY OF TECHNOLOGY



UNIVERSITY OF GOTHENBURG



HARBORFLOW LOGISTICS AB

Assignment 2

Port Intelligence Console

Turn fragmented port data into operational decisions

COURSE DIT014 - Fundamentals of Programming | HT26 | Group assignment: 3-4 students

Teacher: Björn Praestegaard Larsen
Email: bjornpr@chalmers.se

Version date: 19 September 2026

The client commission



HarborFlow has proved that its dispatch teams can work from one reliable console. The next problem is larger: vessel schedules, customer bookings, cargo manifests, incident reports and weather forecasts are stored in separate files maintained by different departments. Operations managers spend each morning copying data between spreadsheets before they can answer basic questions.

Your team is commissioned to build a terminal-based Port Intelligence Console. It must load the supplied JSON and CSV dataset, connect records through shared identifiers, generate persistent business outputs and help staff investigate operational risk. The client expects an auditable product: exact output, predictable file handling and errors that inform the user instead of crashing the shift.

CLIENT VOICE "We do not need another dashboard mock-up. We need a dependable tool that can explain what is arriving, what is at risk and what evidence supports the answer." - Karin Holm, fictional HarborFlow Operations Director

Learning goals

- Read, validate and combine JSON and CSV data from a realistic directory structure.
- Use dictionaries and lists to model relationships between vessels, port calls, shipments and incidents.
- Create and update persistent CSV/JSON outputs without losing existing records.
- Use exceptions deliberately around file, parsing and conversion boundaries.
- Apply regular expressions, sequence algorithms and an inverted index to operational text.
- Organize a multi-file Python product with reusable functions and clear separation of concerns.

Dataset and project setup

STARTER PACKAGE Download and extract the Assignment 2 starter package from the Canvas assignment. Keep its directory structure unchanged and run the program from the extracted project root. The included DATA-DICTIONARY.md defines required and optional fields. Intentional malformed and missing records are part of the task and must not be repaired or deleted.

```
harborflow-project/
  main.py
  data_access.py
  operations.py
  text_tools.py
  readme.txt
  test-ledger.csv
  START-HERE.txt
  dataset/
    DATA-DICTIONARY.md
    vessels/          # one JSON file per vessel
    manifests/        # one JSON file per vessel call
    shipments/         # one JSON file per shipment
    incidents/         # one JSON file per incident report
    port_calls.csv
    weather.csv
    exports/
    README.txt
```

Data relationships

Source	Key fields	Connection
vessels/*.json	vessel_id, name, imo, capacity_teu	vessel_id links vessels to manifests and port calls
manifests/*.json	call_id, vessel_id, shipment_ids	shipment_ids link a call to shipment files
shipments/*.json	shipment_id, customer, destination, weight_kg, priority	independent cargo records referenced by manifests
incidents/*.json	incident_id, call_id, title, report	call_id links reports to a port call
port_calls.csv	call_id, vessel_id, port_code, arrival_date	port_code and date link to weather.csv
weather.csv	port_code, date, wind_kmh, precipitation_mm, wave_height_m	forecast evidence for a scheduled call

STARTING ADVICE Inspect several files before coding. Sketch which identifiers connect the datasets, note missing or optional fields, and decide which loader function owns each format. Do not scatter open() calls throughout menu branches.

Permitted scope

Use Python's standard library, including json, csv, re, pathlib and datetime. Third-party packages such as pandas are not permitted. The Assignment 2 starter package supplies the development dataset, and CodeGrade may test with additional or replacement records; never hard-code the sample records shown here.

Assessment and delivery standard

Result	Assessment decision
G - Pass	All Tasks 0-9 satisfy the functional behavior, exact interface, dataset rules, exception requirements and handover criteria. Every member can explain the integrated product.
U - Fail	One or more compulsory requirements are missing, unreliable, outside the permitted scope or cannot be demonstrated as a shared team deliverable.

ONE PASS STANDARD Assignment 2 is assessed G/U. There are no 3/4/5 tiers and every listed task is compulsory. Automated tests provide evidence; the examiner makes the assessment decision.

CodeGrade deliverables

- Create one CodeGrade group containing exactly 3-4 students.
- Submit main.py and every Python module imported by the product.
- Submit readme.txt with member names, concrete contributions, architecture and known limitations.
- Submit test-ledger.csv with the required normal, boundary, missing-data and malformed-data test evidence.
- Do not submit altered copies of the supplied dataset unless CodeGrade explicitly requests them.
- Prompts, punctuation, capitalization, ordering, filenames and output formats are part of the client contract.

Definition of done

- The console starts from main.py, returns after each service and closes only through option 10.
- File paths work from the project root and do not contain a student's absolute path.
- Expected input errors are handled without hiding programming defects.
- Generated files are valid CSV/JSON and can be loaded again on a later run.
- The team has tested normal, boundary, missing-data and malformed-data cases.

Task 0: Build the product shell

BUSINESS OUTCOME Give port staff one stable entry point and protect the shift from file and input failures.

```
HARBORFLOW PORT INTELLIGENCE
1. List registered vessels
2. Inspect a vessel manifest
3. Identify priority cargo
4. Export a customer operations profile
5. Find port calls by month
6. Sanitize an incident report
7. Analyze the longest stable event sequence
8. Assess weather risk for upcoming calls
9. Search incident reports
10. Close console
Select service:
```

- Use a loop-state variable; do not use exit() and do not stop with break.
- Reject non-integer and out-of-range menu input with Error - Select a service from 1 to 10.

- Catch `FileNotFoundException`, `json.JSONDecodeError`, `csv.Error` and `ValueError` only where your product can give a useful response.
 - After a recoverable error, return safely to the menu. Do not use a blanket except clause.
- Console closed. HarborFlow operational data remains safe.

Task 1: List registered vessels

BUSINESS OUTCOME Give coordinators a trustworthy inventory built from independent JSON records.

Read every .json file in `dataset/vessels`. Validate that `vessel_id`, `name`, `imo` and `capacity_teu` exist. Sort valid vessels by name, case-insensitively, and print one line per vessel. A malformed file or a record missing a required field must be reported and skipped without preventing valid vessels from appearing.

```
Registered HarborFlow vessels:
- Baltic Pioneer | IMO 9312048 | 824 TEU
- Nordic Relay | IMO 9471155 | 1,120 TEU
- Skagerrak Star | IMO 9684207 | 1,460 TEU
Skipped vessel files: 1
```

CODE STARTER Write `load_json(path)` for one file and `load_vessels(directory)` for the collection. A useful collection loader can return both valid records and a count or list of rejected files.

Task 2: Inspect a vessel manifest

BUSINESS OUTCOME Connect a scheduled port call to its vessel and cargo instead of reading files in isolation.

Display registered vessels, ask for a vessel name and a reference date in YYYY-MM-DD format. Remove surrounding spaces from the name and match it case-insensitively. Filter port calls to that vessel, then locate the earliest call on or after the reference date. Load the matching manifest through `call_id`, then resolve every `shipment_id` against `dataset/shipments`. Print the call, totals and cargo lines in manifest order. A missing shipment is listed as MISSING RECORD and counted separately. If several calls share the earliest date, choose the call with the alphabetically smallest `call_id`. For an invalid date, print Error - Date must use YYYY-MM-DD. and repeat only the date prompt. For an unknown vessel, print Vessel not found. If no qualifying call exists, print No upcoming port calls found. If the selected call has no readable valid manifest, print Manifest unavailable. In these three cases, return to the menu.

```
Vessel name: Nordic Relay
Manifest for Nordic Relay | Call HFL-GOT-2048 | 18 October 2026
1. SHP-1042 | Norra MedTech AB | 860.0 kg | REFRIGERATED
2. SHP-1051 | MISSING RECORD
Declared shipments: 2
Resolved weight: 860.00 kg
Missing shipment records: 1
```

- Do not assume filenames equal business identifiers; read and compare fields.
- Use `datetime` to present ISO dates as day month year.
- Treat hazardous, temperature_controlled and cargo_type as optional shipment fields. Missing boolean flags count as false. When `cargo_type` is printed in the manifest, print NOT PROVIDED if it is absent.

Task 3: Identify priority cargo

BUSINESS OUTCOME Turn raw shipment attributes into a documented operational decision.

Ask for a call ID, remove surrounding spaces and match it case-insensitively. For that call, classify every resolved shipment using the first matching rule below. Print the category and reason in manifest order, then summarize counts by category in the stated order. If the call is unknown, print Port call not found. If its manifest is missing or malformed, print Manifest unavailable. In either case, return to the menu.

First matching condition	Category
hazardous is true	CONTROLLED
temperature_controlled is true	COLD CHAIN
priority is EXPRESS or weight_kg >= 5000	EXPEDITE
otherwise	STANDARD

```
SHP-1045 | EXPEDITE | express service
SHP-1046 | CONTROLLED | hazardous cargo
SHP-1067 | EXPEDITE | express service
Category totals: CONTROLLED=1, COLD CHAIN=0, EXPEDITE=2, STANDARD=0
```

ENGINEERING NOTE Make classification a pure function that receives one shipment dictionary and returns both category and reason. Rule order is business policy and must be visible in the code.

Task 4: Export a customer operations profile

BUSINESS OUTCOME Give account managers one persistent CSV record they can open outside the console.

Ask for a customer name, remove surrounding spaces, and match customer names case-insensitively across all valid shipment files. Create exports/customer-profiles.csv with the exact header below. If the customer is new, append one row. If the customer already exists, replace that row without changing other customer rows or creating a duplicate. If no shipment matches, print Customer not found. and do not create or modify the export file.

```
customer,shipment_count,total_weight_kg,express_count,destinations
Norra MedTech AB,8,6420.50,3,CPH|GOT|OSL
```

- Write the header exactly once, including when the file is first created.
- Sort unique destination codes and join them with |.
- Use a temporary in-memory list and rewrite safely when updating an existing row.
- Format total_weight_kg with exactly two decimals.

```
Customer name: Norra MedTech AB
Customer profile updated: exports/customer-profiles.csv
```

Task 5: Find port calls by month

BUSINESS OUTCOME Help berth planners review seasonal workload across all scheduled calls.

Ask for a month as an integer from 1 to 12. For non-integer or out-of-range input, print Error - Enter a month from 1 to 12. and repeat only the month prompt. Read port_calls.csv, resolve each vessel name, and print calls in that month sorted by arrival_date and then vessel name. Report and skip invalid dates. If a row refers to an unknown vessel_id, report and skip that row. If no valid calls match, print No port calls found.

```
Month (1-12): 10
Port calls in October:
- 02 October 2026 | Baltic Pioneer | CPH | HFL-CPH-2017
- 18 October 2026 | Nordic Relay | GOT | HFL-GOT-2048
- 18 October 2026 | Skagerrak Star | AAR | HFL-AAR-2071
```

BOUNDARY THINKING Validate the user's month separately from dates inside the dataset. One bad row must not invalidate the entire planning report.

Task 6: Sanitize an incident report

BUSINESS OUTCOME Protect commercially sensitive identifiers before a report is shared with an external partner.

Display incident titles sorted case-insensitively and numbered from 1. Ask for the list number; for invalid input, print Error - Select a listed incident. and repeat only that prompt. Using regular expressions, replace every container identifier, seal number and email address in the selected report. Preserve all other text, punctuation and line breaks. Save the result in exports/sanitized/<incident_id>-sanitized.txt.

- A container identifier is four letters followed by seven digits. Match letters case-insensitively.
- A seal number is SEAL- followed by six digits. Match the SEAL prefix case-insensitively.
- For this assignment, an email contains letters, digits or ._%+- before @, a domain containing letters, digits, dots or hyphens, and a final dot followed by at least two letters.

Sensitive value	Input pattern example	Replacement
Container ID	MSCU1234567	[CONTAINER]
Seal number	SEAL-884210	[SEAL]
Email address	name@company.se	[EMAIL]

```
Container MSCU1234567 arrived with seal SEAL-884210. Notify ops@harborflow.se.
```

```
[CONTAINER] arrived with seal [SEAL]. Notify [EMAIL].
```

- Use re.sub() for the three transformations.
- A report is considered changed when at least one substitution occurs.
- If nothing changes, do not create a file; print Incident contains no share-sensitive identifiers.
- Create the output directory when necessary.

Task 7: Analyze the longest stable event sequence

BUSINESS OUTCOME Find the longest uninterrupted stretch of distinct operational events in a vessel journey.

Each manifest contains an event_codes list in chronological order. For a selected call, find the length and content of the longest contiguous sequence that contains no repeated event code. When two sequences have equal maximum length, keep the first. Implement the sliding-window logic yourself; do not use third-party libraries.

```
Event codes: GATE_IN,SCAN,WEIGH,SCAN,LOAD,SEAL,DEPART
Longest stable sequence length: 5
Sequence: WEIGH -> SCAN -> LOAD -> SEAL -> DEPART
```

HOW TO THINK Track the start of the current window and the most recent position of each event in a dictionary. When an event repeats inside the active window, move the start just beyond its previous position; do not restart blindly from the repeated event.

Task 8: Assess weather risk for upcoming calls

BUSINESS OUTCOME Join schedules and forecasts so a planner can decide whether extra controls are needed.

Read port_calls.csv and weather.csv. Ask for a vessel and a reference date in YYYY-MM-DD format. Remove surrounding spaces from the vessel name and match it case-insensitively. An upcoming call is one whose arrival_date is on or after that date. Match each qualifying call by port_code and arrival_date. Print forecasts chronologically, using call_id as the tie-breaker, and apply every recommendation whose condition is true.

- If the vessel is unknown, print Vessel not found. and return to the menu.
- If the date is invalid, print Error - Date must use YYYY-MM-DD. and repeat only the date prompt.
- Report and skip an invalid port-call date without stopping later valid calls.
- If no valid upcoming calls exist, print No upcoming port calls found.

Condition	Recommendation
wind_kmh >= 45	Suspend crane operations.
wave_height_m >= 2.5	Arrange tug assistance.
precipitation_mm >= 8 and wind_kmh < 45	Prepare covered handling.
all three values below thresholds	Normal operating plan.

```
Nordic Relay | GOT | 18 October 2026
Wind: 48.0 km/h | Rain: 3.2 mm | Waves: 2.7 m
Recommendation: Suspend crane operations. Arrange tug assistance.
```

- Recommendations can be combined and must appear in table order.
- A missing forecast prints Forecast unavailable and does not stop later calls.
- Convert numeric CSV fields inside a focused try block and report malformed forecast rows.

Task 9: Search incident reports

BUSINESS OUTCOME Build a reusable evidence index so operations can retrieve prior incidents by keywords.

Normalize all incident report text to lowercase words, removing punctuation with regex. Build an inverted index whose keys are words and whose values are lists of incident IDs containing that word. Also build a title lookup whose keys are incident IDs and whose values are incident titles. Save both structures in dataset/incident-index.json and load the cached file on later runs.

```
{
  "index": {
    "delay": ["INC-087", "INC-119"],
    "reefer": ["INC-102", "INC-119"],
    "temperature": ["INC-102"]
  },
  "titles": {
    "INC-087": "Gate congestion delay",
    "INC-102": "Reefer temperature deviation",
    "INC-119": "Reefer transfer delayed"
  }
}
```

Normalize the user's query with the same rules. Score an incident by the number of distinct query words whose index list contains its ID. Print only positive matches, sorted by descending score and then incident ID. Use the cached title lookup for the displayed title.

```
Search incident reports: reefer delay
Matches for 'reefer delay':
- INC-119 | score 2 | Reefer transfer delayed
- INC-087 | score 1 | Gate congestion delay
- INC-102 | score 1 | Reefer temperature deviation
```

- Do not add an incident ID twice to one word's list when a word repeats in a report.
- Write valid JSON with the top-level keys index and titles. Sort word keys, title keys and every incident ID list.
- If the cache is missing, malformed or older than any source incident file, rebuild it; otherwise reuse it.
- Searching must use the loaded index and title lookup rather than rereading incident reports.

Engineering and handover

HarborFlow receives one product, not a collection of task scripts. Agree on interfaces before dividing work, integrate continuously and ensure each member can trace a question from menu input through loaders, business logic and output.

Suggested architecture

```
main.py           # menu and orchestration
data_access.py    # JSON/CSV loading, validation and paths
operations.py     # manifests, classification, forecasts and exports
text_tools.py     # regex sanitization, sequence and search index

if __name__ == "__main__":
    main()
```

STARTER, NOT A SOLUTION You may adapt these boundaries. Keep user interaction, file access and business rules separate enough that each can be tested and explained.

Recommended team workflow

1. Inventory the supplied files and agree on a shared data dictionary before implementing features.
2. Build and review loaders first; every later service depends on reliable, consistent records.
3. Assign initial feature ownership, then require a second member to review its normal and failure paths.
4. Integrate into main.py continuously and keep a shared test ledger with input, expected result and actual result.
5. Run a final client demonstration from a clean folder with a fresh copy of the dataset.

Product deliverables

Deliverable	Client expectation
Python product	One integrated, runnable console with all required modules
Generated evidence	Correct CSV, JSON and sanitized text outputs created only when required
readme.txt	Team, architecture, run instructions, contributions and known limitations
test-ledger.csv	Normal, boundary, malformed-data and missing-link cases recorded by the team

Acceptance test campaign

Area	Minimum evidence
Files	Missing file, malformed JSON, invalid CSV number, mixed valid/invalid records
Relationships	Known ID, missing linked shipment, unknown vessel/customer
Export	First write, second customer, update existing row, reload generated CSV
Regex	Each identifier type alone, several in one line, no match
Sequence	All unique, immediate repeat, repeated event outside active window, tie
Weather	Each threshold, combined rules, absent forecast
Search	Single word, repeated query word, no match, corrupt cache rebuild

CodeGrade preflight

- Run the submitted project from a clean folder using only relative paths.
- Compare prompts, output order and generated filenames character by character.
- Remove debugging prints, temporary test data and machine-specific paths.
- Confirm all Python modules and readme.txt are included in the group attempt.
- Review CodeGrade feedback and resubmit before the deadline when allowed.

Final client handover checklist

- All Tasks 0-9 are integrated and reachable from the menu.
- Generated CSV and JSON files remain valid after repeated runs.
- Expected data problems produce useful messages; programming errors are not silently swallowed.
- Functions, names, modules and comments make the design maintainable.
- The product minimizes exposed customer identifiers, preserves auditable evidence and avoids unnecessary repeated file processing.
- Every group member can demonstrate the system and explain one end-to-end data flow.
- The final CodeGrade submission is recorded under the correct group.